



Pashov Audit Group

Pare Security Review

September 23rd 2026 - September 24th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. Executive Summary	4
6. Findings	4
Low Severity	5
[L-01] Caller-selected vault can retain stock spending authority	5
[L-02] Caller-supplied vault can redirect stock refunds to an arbitrary token	6
Info Severity	8
[I-01] DividendLP mints pToken liquidity positions against arbitrary tokens instead of the intended USDG	8
[I-02] YT holders lose accrued yield because a classifier can stamp one dividend after maturity	9
[I-03] PT claims can exceed backing because guardian reattribution can decrease the dividend index	11
[I-04] YT holders lose pre-maturity dividends because stale split classification falsely advances finality	12
[I-05] PT holders lose pre-series value because new vaults snapshot a pre-fold index	14
[I-06] Zero-liquidity ternary selection bypasses withdrawal minima and burns the position NFT	15



1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Telegram [@pashovkrum](#).

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical

Loss or compromise of critical assets, or protocol-wide impact.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High - leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium - leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low - leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium - only a conditionally incentivized attack vector, but still relatively likely



Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of Pare was conducted by the Pashov Audit Group team. This report documents 8 issues across the reviewed codebase, reported by 0x37, 0xApple, Auditor-Nate, Digital Forensic, farismaulana, Mladenov, Playboi, Valves Security.

SCOPE

"src/periphery/" — one contract, DividendLP.sol.

REVIEW COMMIT HASH

[f14bb7164ccc24d20a3efa5e557f300e0e79af90](#)

FIXES REVIEW COMMIT HASH

[51542756b56a29b0111e5d72884cc6ef5042bd4d](#)

6. Findings

ID	TITLE	SEVERITY	STATUS
L-01	Caller-selected vault can retain stock spending authority	Low	Fixed
L-02	Caller-supplied vault can redirect stock refunds to an arbitrary token	Low	Fixed
I-01	DividendLP mints pToken liquidity positions against arbitrary tokens instead of the intended USDG	Info	Fixed
I-02	YT holders lose accrued yield because a classifier can stamp one dividend after maturity	Info	Fixed
I-03	PT claims can exceed backing because guardian reattribution can decrease the dividend index	Info	Fixed
I-04	YT holders lose pre-maturity dividends because stale split classification falsely advances finality	Info	Fixed



I-05	PT holders lose pre-series value because new vaults snapshot a pre-fold index	Info	Fixed
I-06	Zero-liquidity ternary selection bypasses withdrawal minima and burns the position NFT	Info	Fixed

Low Severity

[L-01] Caller-selected vault can retain stock spending authority

Low

Fixed

LOCATION

DividendLP.sol:deposit:95-97

SUMMARY

`DividendLP` is meant to use the caller-selected vault only for the current stock split and then return to a stateless condition. It clears the approvals it granted to the position manager and quote token, but it never clears the approval granted to `p.vault`.

This leaves a persistent allowance over the periphery's future stock balance:

```
stock.safeTransferFrom(msg.sender, address(this), p.stockAmount);
stock.forceApprove(address(p.vault), p.stockAmount);
uint256 minted = p.vault.split(p.stockAmount);
```

Because the vault address is caller-selected, an attacker can supply an implementation whose `split()` leaves the allowance unused, so the approval survives the call. This is an authorization-boundary failure, not merely cosmetic allowance residue.

IMPACT

A malicious vault can reuse an approval created during one deposit to drain stock the contract holds later, including stock belonging to subsequent users or integrations.

EXPLOIT PATH

1. The attacker calls `deposit()` with a malicious vault whose `stock()` returns the legitimate stock token and whose `split()` leaves the approval unused.
2. `DividendLP` approves the malicious vault for the requested stock amount and completes the deposit without resetting that approval.
3. A later user deposits the same stock through the normal flow, leaving stock in `DividendLP` during processing or as a subsequent balance.
4. The malicious vault calls `stock.transferFrom(DividendLP, attacker, amount)` and transfers the approved stock away.



PREREQUISITES

A malicious vault-compatible contract must be selected for an initial deposit and must leave its stock allowance unused, and `DividendLP` must later hold the same stock token. No privileged role is required.

RECOMMENDATION

Revoke the allowance immediately after the external split call:

```
stock.forceApprove(address(p.vault), p.stockAmount);
uint256 minted = p.vault.split(p.stockAmount);
stock.forceApprove(address(p.vault), 0);
```

Additionally, enforce a trusted-vault allowlist or registry before any token transfer or approval:

```
if (!trustedVault[address(p.vault)]) revert InvalidVault();
```

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/70dc82a>

The stock allowance to the caller-selected vault is now reset to 0 immediately after `split()`, as recommended:

```
stock.forceApprove(address(p.vault), p.stockAmount);
uint256 minted = p.vault.split(p.stockAmount);
stock.forceApprove(address(p.vault), 0);
```

Added a regression test, `test_deposit_leavesNoStockAllowanceOnTheVault`. It uses a vault that leaves the allowance unused and then tries to pull stock from `DividendLP`. The test fails before the fix and passes after. The full suite passes (168 tests).

We haven't added the vault allowlist. `DividendLP` is permissionless by design: it works with any `StripVault` series without an admin, and it holds no balances between calls. With the allowance reset, a caller-selected vault keeps no spending authority once the call ends. Happy to add a registry check if you see a remaining path.

[L-02] Caller-supplied vault can redirect stock refunds to an arbitrary token

Low

Fixed

LOCATION

`DividendLP:withdraw:146-185`

SUMMARY

The periphery is meant to unwind a PT/quote position and return either merged stock or residual PT to the position owner. The stock refund should therefore always use the underlying stock token tied to the validated PT/YT series.

Instead, `withdraw()` reads the stock token from the caller-selected vault right before refunding and trusts whatever it returns. Nothing links that address to the PT/YT tokens used in the same withdrawal or to a trusted vault configuration:



```
IERC20 stock = p.vault.stock();
_refund(stock, stockOut);
_refund(pt, ptOut);
_refund(p.quote, quoteOut);
```

A malicious vault can return an unrelated ERC-20, so a positive `stockOut` is paid out of DividendLP's balance of that unrelated token. The same trust gap lets a malicious vault return arbitrary stock, PT, and YT addresses, and it also affects the deposit path, where such a vault can return an arbitrary stock token and receive an allowance before `split` is invoked.

IMPACT

A caller can have `stockOut` refunded in any ERC-20 they choose by supplying a matching vault. If DividendLP holds that token from another operation or integration, the attacker drains it while the real PT/YT withdrawal proceeds are accounted for separately.

This breaks asset isolation and can expose balances left behind by other operations or integrations.

EXPLOIT PATH

1. The attacker supplies a vault whose `stock()` returns token X and whose PT/YT pair is compatible with an NFT position they own.
2. They call `withdraw()` with that vault, the NFT, and `ytToMerge` greater than zero.
3. `_unwind()` obtains PT from the position and the vault merge path increases `stockOut` by the merged amount.
4. `withdraw()` calls `_refund()` on token X and transfers `stockOut` units of X from DividendLP to the attacker.

PREREQUISITES

The attacker must control or supply a vault whose `stock()` getter returns the target token and must own an NFT whose PT/quote pair passes `_unwind()`. DividendLP must hold enough of the returned token for the resulting `stockOut` refund to succeed.

RECOMMENDATION

Validate the token relationships before any asset movement and use the validated underlying token for refunds:

```
IERC20 stock = p.vault.stock();
IERC20 pt = IERC20(address(p.vault.pt()));
IERC20 yt = IERC20(address(p.vault.yt()));
require(address(stock) == trustedStockForSeries[address(p.vault)]);
```

Also validate the PT/YT series and vault registration before continuing. A stronger fix is to maintain an immutable or governance-controlled registry of approved vaults and their stock/PT/YT addresses, rejecting arbitrary vault contracts in both `deposit()` and `withdraw()`.



RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/8595696>

DividendLP no longer trusts amounts reported by the caller-selected vault. It pays out only what the vault actually delivered during the call, measured by balance change:

`withdraw()`: `stockOut` is the increase in the stock balance across merge/redeem, so a vault naming an unrelated token cannot have DividendLP's existing balance of it paid out.

`deposit()`: the pTokens placed and yield tokens returned are the balance increases across `split()`, so the same applies to `pt()` and `yt()`.

A vault naming one token as both `pt()` and `yt()` is rejected, since it would be counted twice.

Added regression tests `test_withdraw_paysOnlyStockTheVaultDelivered`, `test_deposit_paysOnlyYieldTokensTheSplitDelivered` and `test_deposit_rejectsVaultNamingOneTokenAsBoth`. The first two fail before the fix. The full suite passes (171 tests).

We kept DividendLP permissionless instead of adding a vault registry: it works with any StripVault series without an admin, and a registry an owner can edit could block withdrawals from a delisted vault. With balance-based accounting, a malicious vault can only move tokens it delivered itself. Happy to revisit if you see a remaining path.

Info Severity

[I-01] DividendLP mints pToken liquidity positions against arbitrary tokens instead of the intended USDG

Info

Fixed

SUMMARY

`quote` is documented as "the pool's other token (USDG)" (DividendLP.sol:27) but is a free caller-supplied field in both parameter structs with no validation.

The constructor pins only the position manager (DividendLP.sol:72-74), `deposit` places liquidity in whatever token is passed (DividendLP.sol:99-104,121-137), and `withdraw`'s `WrongPool` check only matches the position's existing pair (DividendLP.sol:181).

IMPACT

The contract cannot guarantee the positions it mints sit in the intended pToken/USDG pools. Any ERC-20 can become the pair, fragmenting liquidity into unintended pools.

RECOMMENDATION

Pin the quote token at construction and enforce it in both entrypoints:

```
+ IERC20 public immutable quote; // USDG
+
+ constructor(INonfungiblePositionManager npm_, IERC20 quote_) {
+     npm = npm_;
+     quote = quote_;
+ }
```

Then either drop the `quote` field from both parameter structs and use the immutable in `_mint` and `_unwind`, or keep the field and revert when `p.quote != quote`.

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/0319930>

The quote token is now pinned at construction (`IERC20 public immutable quote, USDG`), and both `deposit()` and `withdraw()` revert with `WrongQuote` when `p.quote` differs. We kept the quote field in both parameter structs so existing callers keep the same ABI. Added `test_rejectsQuoteOtherThanTheConfiguredOne`, covering both entry points. The full suite passes (172 tests).

[I-02] YT holders lose accrued yield because a classifier can stamp one dividend after maturity

Info

Fixed

SUMMARY

For a single noncollapsed pending change, `MultiplierAccountant::sync` records the token's correct dividend effective time in `pending.timestamp` (`MultiplierAccountant.sol:248-259,612-632`). `classifyDividend` uses that recorded time, but `classifyDividends` also accepts a single leg and lets `CLASSIFIER_ROLE` supply any timestamp between `pendingWindowStart` and `pendingObservedAt` (`:287-296,309-335`). It never requires that leg to use the known `pending.timestamp`.

Consider one 2% dividend scheduled and announced before a vault's maturity. A pre-effective `sync()` observes the schedule while the old multiplier still applies. The dividend becomes effective before maturity, but the next `sync()` runs after maturity: its pending record is noncollapsed with the correct pre-maturity `pending.timestamp`, while the observation window spans maturity. The classifier then calls `classifyDividends([1.02e18], [tLate])` with `tLate` after maturity but still inside that window. The ratio and window checks pass, and the fold is scheduled at the classifier's substituted time.

Before the fold's `guardianDelay` expires, `StripVault::settle` calls `sync()` and then `indexFinalAt(maturity)` (`StripVault.sol:156-168`). Because `indexFinalAt` checks the supplied timestamp (`MultiplierAccountant.sol:492-495`), the waiting fold appears to lie entirely after maturity, so settlement freezes the old `dT`. Applying the fold later cannot change that frozen value.

IMPACT

With `d0 = 1e18`, 102 PT and 102 YT backed by 102 raw stock units after issuance fees, the true 2% pre-maturity dividend makes the correct `dT = 1.02e18`. PT holders should receive 100 units and YT holders 2 units of gross yield; at a 5% yield fee, YT holders receive 1.9 units and the treasury 0.1.

The wrong-side timestamp instead freezes `dT = 1e18`: PT receives all 102 units, while YT holders and the yield-fee recipient receive zero (`StripVault.sol:171-190`).



The allocation is permanent within the settled vault, and a YT-only holder cannot merge without obtaining PT. An honest classifier that copies `pending.timestamp` avoids the loss.

EXPLOIT PATH

1. A 2% dividend is scheduled and announced before the vault's maturity, and becomes effective before maturity.
2. A pre-effective `sync()` observes the schedule under the old multiplier; the next `sync()` runs after maturity, leaving a noncollapsed pending record whose window spans maturity.
3. The classifier calls `classifyDividends([1.02e18], [tLate])` with `tLate` after maturity but inside the window, so the fold is scheduled at that substituted time.
4. `StripVault::settle` runs `sync()` then `indexFinalAt(maturity)` before `guardianDelay` expires, freezing `dt = 1e18` since the fold appears to lie after maturity.

PREREQUISITES

The caller needs `CLASSIFIER_ROLE`. The supplied README explicitly asks reviewers to assess what a compromised classifier holding only that role can do ([README.md:82-85](#)).

RECOMMENDATION

For a one-leg, noncollapsed pending change, bind `classifyDividends` to `pending.timestamp`, or reject that call shape and require `classifyDividend`. For composites, require authenticated per-action effective times before writing checkpoints. Do not let a classifier move an already known effective time across a series maturity merely by supplying another in-window timestamp.

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/9a79ef5>

We took the first option. For a single observed (not collapsed) pending change, `classifyDividends` now accepts only one leg stamped at `pending.timestamp`; anything else reverts with `LegTimestampMismatch`. The classifier can no longer move a known effective time elsewhere in the window, including across a series maturity. Collapsed composites keep the existing per-leg checks: in-band legs, ordered timestamps inside `[pendingWindowStart, pendingObservedAt]`, and a product matching the observed ratio.

On authenticated per-action times for composites: the token exposes only the latest update's effective time, so there is no on-chain source for earlier legs. The window bound is the tightest check available. Composites whose legs can't be rebuilt from the token's events go to the guardian path.

Added `test_classifyDividends_singleChangeMustUseItsRecordedTimestamp`. It covers a late in-window timestamp, a split into two legs, and the accepted case at the recorded time. The full suite passes (173 tests).



[I-03] PT claims can exceed backing because guardian reattribution can decrease the dividend index

Info

Fixed

SUMMARY

`MultiplierAccountant::classifySplit` accepts a clean reverse split such as 1:2 and schedules a Split fold (`MultiplierAccountant.sol:380-396,536-546`). During that fold's reaction window, `reattributeFoldAsDividend` lets `GUARDIAN_ROLE` change its kind to Dividend and immediately apply it (`:360-371`).

The Dividend branch of `_applyFold` multiplies `dividendIndex` by the observed new-to-old multiplier ratio, without checking that the ratio is at least `1e18` or that the resulting index stays nondecreasing (`:575-603`).

Concretely, a funded vault has `d0 = 1e18` . The stock multiplier halves in a clean 1:2 reverse split effective before maturity, and a classifier correctly calls `classifySplit(1, 2)` , leaving the dividend index unchanged while the Split fold waits. Before the deadline, a guardian calls `reattributeFoldAsDividend()` , which applies the halving ratio and writes a `0.5e18` dividend checkpoint at the earlier effective time. Once maturity passes, `StripVault::settle` freezes `dT = 0.5e18` ; it checks finality but not `dT >= d0` (`StripVault.sol:156-168`).

The guardian is authorized to override the kind during this window and immediate application is intended, but the override can place the vault's unsigned redemption formulas outside their valid index domain.

IMPACT

With `d0 = 1e18` and `dT = 0.5e18` , `redeemPT` pays two raw stock units per PT, while `redeemYT` reverts on the checked subtraction `dT - d0` (`StripVault.sol:171-190`).

If 100 PT and 100 YT are backed by 100 raw stock units after issuance fees, the first 50 PT redeemed can drain the vault. Later PT holders cannot be paid, and YT-only holders cannot redeem at all.

A correct Split classification would leave `dT = d0` , paying one unit per PT and no dividend yield. Holders who own both PT and YT can still merge while backing remains; there is no escape for unmatched holders.

EXPLOIT PATH

1. A classifier correctly classifies a clean reverse split via `classifySplit(1, 2)` , scheduling a Split fold and leaving `dividendIndex` unchanged.
2. Before the fold's deadline, `GUARDIAN_ROLE` calls `reattributeFoldAsDividend()` , changing the kind to Dividend.
3. `_applyFold` multiplies `dividendIndex` by the sub-1e18 ratio, writing `dT = 0.5e18` at the earlier effective time.
4. After maturity, `settle` freezes the invalid index; PT redemptions overpay and drain backing, and YT redemptions revert.

PREREQUISITES

Requires `GUARDIAN_ROLE` to reattribute against the true reverse-split classification. This is a privileged action; the source does not establish an unprivileged route.



RECOMMENDATION

Require any Dividend reattribution to preserve a positive, nondecreasing `dividendIndex`, and reject a reverse-ratio fold before changing its kind.

Apply the same index-domain rule to every accountant writer that can create a Dividend checkpoint, and reject `dT < d0` at vault settlement as a final safeguard.

If negative dividend adjustments are intended, define backing and redemption formulas that can represent them without overclaims or subtraction failure.

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit//7cc77d6>

The dividend index is now non-decreasing at every writer:

`reattributeFoldAsDividend` reverts with `NotADividend(r)` when the fold's ratio is below `1e18`, before changing its kind, so a reverse split can no longer be reattributed as a dividend.

`executeResolution` applies the same rule to each Dividend leg. It is the only other writer of dividend checkpoints; the classifier paths (`classifyDividend`, `classifyDividends`) already require `r > 1e18`.

As a final safeguard, `StripVault.settle` reverts with `IndexBelowStart` if `dT < d0`, keeping redemptions inside their valid domain. Merge remains available as the exit.

Negative dividend adjustments aren't intended; a decrease is always a reverse split and goes to `splitFactor`.

Added `test_reattributeFoldAsDividend_rejectsReverseRatio`,

`test_guardian_dividendLegCannotDecreaseIndex` and `test_settle_refusesIndexBelowStart`. The first two fail before the fix. The full suite passes (178 tests).

[I-04] YT holders lose pre-maturity dividends because stale split classification falsely advances finality

Info

Fixed

SUMMARY

`MultiplierAccountant::classifySplit` classifies a stored pending change without first refreshing the token's multiplier-update metadata (`MultiplierAccountant.sol:385-396,525-529`). A current `sync()` can mark a pending change collapsed once `effectiveAt` advances, even when the multiplier returns to the pending endpoint (`:211-218,262-269`). That detection is skipped when the classifier runs before another `sync()` .

`_finalize` then calls `_recordClean` , which treats equality between the live multiplier and the stored endpoint as proof that no further update happened, and moves `lastCleanSyncAt` to the classification time (`:548-552,606-610`).

Concrete walkthrough. The accountant starts with `uiMultiplier = dividendIndex = 1e18` . A 2:1 split is announced and observed by `sync()` ; after it takes effect a later `sync()` opens a noncollapsed pending `1e18 -> 2e18` split. Before a funded vault's maturity `M`, with no further `sync()` , the issuer applies a 25% reinvested dividend (`2e18 -> 2.5e18`) then a 4:5 reverse split (`2.5e18 -> 2e18`), each with its own later effective time. Their product returns the live multiplier to `2e18` , but correctly `dividendIndexAt(M)` should be `1.25e18` while both splits belong in `splitFactor` .



After **M**, a classifier calls `classifySplit(2, 1)` on the old pending record. The stored 2:1 ratio validates, and `_recordClean` advances `lastCleanSyncAt` to this post-maturity call because the live multiplier is again `2e18`. When `StripVault::settle` later calls `sync()` (after `guardianDelay`), that call detects the later `effectiveAt` and opens a collapsed net-zero pending change, but sets `pendingWindowStart` to the false post-maturity clean time (`:219-236`). `indexFinalAt(M)` then returns true while the pre-maturity dividend is unresolved (`:492-495`), and the vault freezes `dT = 1e18` instead of `1.25e18` (`StripVault.sol:156-168`).

IMPACT

The pre-maturity dividend is permanently misallocated from YT holders to PT holders. Backing is not lost, but yield that should reach YT and the fee recipient goes entirely to PT.

With 100 PT and 100 YT backed by 100 raw stock-token units, correct `d0 = 1e18` and `dT = 1.25e18` pay 80 units to PT and 20 units of gross yield to YT; a 5% YT fee splits that into 19 units for YT holders and 1 unit for treasury. The frozen `dT = 1e18` instead pays all 100 units to PT and zero to YT or the fee recipient (`StripVault.sol:171-193`). A YT-only holder cannot recover via the paired-token merge exit without first acquiring PT.

EXPLOIT PATH

1. A noncollapsed pending `1e18 -> 2e18` split record exists after the split takes effect.
2. Before maturity **M**, the issuer applies a reinvested dividend and a reverse split whose product returns the live multiplier to the pending endpoint, with no intervening `sync()`.
3. After **M**, a classifier calls `classifySplit(2, 1)`; `_recordClean` advances `lastCleanSyncAt` to the post-maturity time because the live multiplier again equals the stored endpoint.
4. `settle` eventually calls `sync()`, which anchors `pendingWindowStart` to the false clean time, making `indexFinalAt(M)` return true.
5. The vault freezes `dT = 1e18`, denying the pre-maturity dividend to YT holders.

PREREQUISITES

Requires an authorized classifier to call `classifySplit` on the stale pending record before any `sync()` collapses it. The failure does not require the classifier to supply a false timestamp or a guardian to misclassify an action; it only requires the intervening `sync()` to be missed. Calling `sync()` before `classifySplit` would have collapsed the original record and blocked ordinary classification.

RECOMMENDATION

Refresh the token's effective-update state before any classification finalizes an open pending record. If `effectiveAt` or an authenticated token update sequence changed since that record was observed, mark it collapsed and require reconstruction of the individual actions and their effective times.

Advance `lastCleanSyncAt` only after the accountant has verified every intervening update was processed; present-value equality alone must not certify a clean history. Keep maturity finality blocked until every possible pre-maturity dividend is represented by a correctly timed checkpoint.



RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/060253c>

`_recordClean` no longer treats the live multiplier equalling the accounted value as proof of a clean history. It now applies the same check `sync()` uses to detect round trips: if a token update has taken effect since the last sync (effectiveAt in the past and different from the last seen one), `lastCleanSyncAt` stays at `pendingObservedAt` instead of moving to the classification time. In the reported path, `classifySplit(2, 1)` still records the real 2:1 split, but the clean time is not advanced. The next `sync()` (including the one in `settle()`) opens the collapsed net-zero composite with a window that still covers the missed dividend and reverse split, so `indexFinalAt(M)` stays false until the guardian resolves it. The same applies to the guardian path, which also finalizes through `_recordClean`.

Added test `_classifySplit_staleRecordDoesNotAdvanceFinality`, which follows the reported scenario. It fails before the fix and passes after. The full suite passes (174 tests).

[I-05] PT holders lose pre-series value because new vaults snapshot a pre-fold index

Info

Fixed

SUMMARY

`MultiplierAccountant::classifyDividend` schedules a dividend fold, then `_finalize` clears `pending` and advances `lastMultiplier` without applying that fold (`MultiplierAccountant.sol:287-296,548-557`). During the guardian reaction window, `isSynced()` returns true whenever the token's current multiplier equals `lastMultiplier`; it never checks `_fold` (`: 485-486`).

`StripVault` trusts that result in its constructor and stores the still-old `dividendIndex` as the immutable `dividendIndex0` (`StripVault.sol:101-111`).

Concretely, an accountant starts at `dividendIndex = 1e18`. A genuine 1% dividend becomes effective and is classified before a new series is created. Classification schedules the fold for the configured guardian delay, leaving the live index at `1e18` while `isSynced()` still reports true. The operator deploys the next vault during this interval, as contemplated by the documented next-series cap-raise path (`StripVault.sol:55-57`), so the new vault records `d0 = 1e18` even though the dividend was already effective before the vault existed.

After the fold applies, the accountant writes a checkpoint at the dividend's earlier effective timestamp (`MultiplierAccountant.sol:575-603`). If no later dividend intervenes, the new series settles with `dT = 1.01e18` (`StripVault.sol:156-168`), treating a pre-creation dividend as yield earned during the series. No guardian error, inaccurate effective timestamp, or missed update is required to trigger this.

IMPACT

Redemption pays `d0/dT` raw stock per PT via `redeemPT`, with `redeemYT` receiving the complementary share minus its yield fee (`StripVault.sol:171-190`).

With `d0 = 1e18` and `dT = 1.01e18`, for 101 PT and 101 YT backed by 101 raw stock units after issuance fees, PT holders receive 100 units and YT holders receive 1 unit of gross yield; a 5% yield fee then splits that unit into 0.95 for YT holders and 0.05 for the treasury. Had the vault anchored the correct post-fold index of `1.01e18`, all 101 units would go to PT holders and none to YT, since the 1% dividend preceded the series.



Settlement freezes `dT`, so later accountant activity cannot repair the vault's immutable `d0` or its settled payout split. The same stale `d0` also drives the open-fold view and settled-oracle effects. Total backing stays intact, but value shifts from PT holders to YT holders and the fee recipient. A holder of both PT and YT can merge one-for-one to exit whole, but an unmatched PT holder cannot use that path without first acquiring YT.

RECOMMENDATION

Require the creation-time dividend index to be final before a new vault can anchor it. Either reject a vault deployment while an unapplied fold can still affect the index at creation, or apply every eligible fold before taking the snapshot.

Use a final historical index for the creation timestamp, and extend the constructor's sync check so it covers pending folds as well as unclassified multiplier changes.

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/801dc6c>

The StripVault constructor now requires `accountant.indexFinalAt(block.timestamp)` in addition to `isSynced()`. `indexFinalAt` returns false while a fold whose first timestamp is at or before the given time is waiting, as well as for pending changes, so a series cannot be created while a classified dividend's fold is still waiting out the guardian delay. `dividendIndex0` is therefore only snapshotted once the index at creation is final, including the pre-creation dividend.

Added `test_constructor_refusesIndexWithUnappliedFold`. Creation reverts during the fold window, and after `applyFold()` it succeeds with `d0 = 1.01e18`. The test fails before the fix. The full suite passes (175 tests).

[I-06] Zero-liquidity ternary selection bypasses withdrawal minima and burns the position NFT

Info

Fixed

LOCATION

`DividendLP.sol:_unwind`

SUMMARY

`withdraw()` forwards the caller's `WithdrawParams` to `_unwind()`, which resolves the liquidity to remove with a ternary that treats `0` as a sentinel meaning "use the whole position":

```
uint128 liquidity = p.liquidity == 0 ? posLiquidity : p.liquidity;
```

For a position that is already empty, `posLiquidity == 0`, so `liquidity` becomes `0`. The `if (liquidity > 0)` branch is skipped, meaning `decreaseLiquidity()` never runs and the caller's `ptMin` / `quoteMin` are never enforced.

That same zero value then satisfies `if (liquidity == posLiquidity)`, so `npm.burn(p.tokenId)` executes regardless of whether `collect()` returned amounts below the requested minima.



IMPACT

An owner can call withdraw on an already-zero-liquidity position with `ptMin` and `quoteMin` set above the collectible amounts. The minima are silently bypassed and the position NFT is burned while the caller receives zero or insufficient PT/quote output.

EXPLOIT PATH

1. The stored Uniswap position has `posLiquidity == 0`.
2. The caller submits `p.liquidity == 0` with `ptMin` / `quoteMin` above the collectible amounts.
3. The ternary sets `liquidity = 0`, so `decreaseLiquidity()` and its min checks are skipped.
4. `liquidity == posLiquidity` holds, so the NFT is burned despite the minima not being met.

RECOMMENDATION

Guard the burn so it only fires when actual liquidity was removed:

```
if (liquidity == posLiquidity && liquidity > 0) npm.burn(p.tokenId);
```

RESOLUTION

Pare

Fixed in <https://github.com/pare-audit/contracts/commit/5154275>

Applied the recommended guard: `_unwind` now burns the position only when this call removed liquidity (`liquidity == posLiquidity && liquidity > 0`). Withdrawing from an already-empty position still collects anything owed, but no longer burns the NFT without `decreaseLiquidity` enforcing the minima.

Added `test_withdraw_zeroLiquidityPositionIsNotBurned`. It empties a position directly, then withdraws with `ptMin/quoteMin` far above the collectible amounts and checks the NFT is kept. The test fails before the fix. The full suite passes (179 tests).